

Reload Ledger Test Execution Guide

Current automated test coverage, execution commands, and operator checks
2026-07-12

- [Reload Ledger Test Execution Guide](#)
 - [Operator Setup](#)
 - [Suite Execution](#)
 - [Coverage Map](#)
 - [Storage API and Business Rules](#)
 - [Renderer Routes, Templates, and Static Scripts](#)
 - [Ballistics](#)
 - [POS Printing](#)
 - [MCP Bridge](#)
 - [Selenium Browser Workflow](#)
 - [Acceptance Checks](#)

Reload Ledger Test Execution Guide

This document describes how to collect, run, and verify the automated tests in the current repository. The suite covers the storage API, renderer routes/templates, static UI behavior, MCP bridge, ballistics service, POS print service, guarded print utility, and the opt-in Selenium browser workflow. The repository currently defines 195 test functions:

Area	Test file	Count
Storage API and business rules	tests/test_api.py	61
Domain helpers	tests/test_domain.py	5
MCP server bridge	tests/test_mcp_server.py	18
Renderer templates and static scripts	tests/test_templates.py	55
Renderer route proxy behavior	tests/test_renderer_routes.py	28
Ballistics service	tests/test_ballistics_service.py	11
Ballistics routing/configuration	tests/test_ballistics_routing_config.py	3
POS print service	tests/test_pos_print_service.py	10
POS print script guardrails	tests/test_pos_print_script.py	3
Selenium browser workflow	tests/e2e/test_357_magnum_workflow.py	1

Operator Setup

	Step	Operator action	Verification
	1	From a terminal, change to the repository root: <code>cd /home/swiseman/repositories/reloading_app</code> .	Current directory contains <code>compose.yaml</code> , <code>pytest.ini</code> , <code>storage_service/</code> , <code>rendering_app/</code> , <code>ballistics_service/</code> ,

		<code>pos_print_service/</code> , and <code>tests/</code> .
2	Check Docker Compose: <code>docker compose version</code> .	Docker Compose prints a version and exits successfully.
3	Optional PDF tool check: <code>pandoc --version</code> , <code>wkhtmltopdf --version</code> , and <code>xelatex --version</code> .	Tools print versions if the operator needs to regenerate rendered documentation.
4	Avoid destructive cleanup unless intended.	Do not run <code>docker compose down -v</code> unless deleting the persistent <code>reloading-data</code> volume is intentional.

Suite Execution

Step	Operator action	Verification
1	Collect tests: <code>docker compose run --rm storage pytest -collect-only -q</code> .	Output lists the API, domain, MCP, renderer, ballistics, POS, and Selenium test modules.
2	Run the standard non-Selenium suite: <code>POS_PRINT_DRY_RUN=true docker compose run --rm storage pytest -q</code> .	Command exits with status <code>0</code> ; the Selenium test is skipped unless <code>--run-selenium</code> is supplied.
3	Start browser services for E2E: <code>POS_PRINT_DRY_RUN=true docker compose --profile selenium up --build -d</code> .	Storage, renderer, ballistics, web, and Selenium services become healthy.
4	Run E2E headless: <code>docker compose run --rm -e APP_BASE_URL=http://web:8080 -e SELENIUM_REMOTE_URL=http://selenium:4444/wd/hub storage pytest --run-selenium tests/e2e -q</code> .	Command exits with status <code>0</code> ; the Selenium workflow passes.
5	Optional visible E2E: add <code>-e SELENIUM_HEADLESS=false -e SELENIUM_SLOW_MS=350</code> to the E2E command, then open <code>http://localhost:7900</code> .	Browser visibly performs registration, inventory, recipe, batch, QA, replacement-lot promotion, Garmin import, container, depletion, and audit workflows.
6	Stop services without deleting data: <code>docker compose down</code> .	Containers stop; the <code>reloading-data</code> Docker volume remains intact.

For a single targeted test, use this command pattern:

```
docker compose run --rm storage pytest <node-id> -q
```

Coverage Map

Storage API and Business Rules

The API suite covers:

- Tenant isolation and structured database error responses.
- Cartridge workflow defaults, current selection, record scoping, and backfill behavior.
- POS print settings, logo upload/delete, explicit batch print endpoint, and dry-run behavior.
- Item category fields, workflow memberships, bullet ballistic metadata, edit locks, and inventory lot counts.
- Inventory lot creation, powder normalization, cost validation, required lot weights, active-lot replacement, opened-date behavior, adjustment/depletion/restoration, and shortage

rollback.

- Recipe creation, UUID identity, exact component roles, duplicate core-component prevention, source upload/stored-file links, public privacy, expected velocity, source-risk acknowledgements, approval gating, and aggregate performance/cost metrics.
- Batch reservation, production QA gates, production loss replacement accounting, consumption, depletion, cost-per-cartridge status, cancellation return/loss accounting, and inventory return validation.
- Garmin Xero C1 Pro FIT import, stored import files, derived performance fields, and editable context fields.
- Container capacity, mixed-batch acknowledgement, assignment quantities, derived storage/depletion state, emptying behavior, and legacy reconciliation.
- Firearm profiles and saved ballistic calculation persistence/user isolation.

Renderer Routes, Templates, and Static Scripts

Renderer tests cover:

- Auth, reset-required handling, logout, readonly/mobile write restrictions, API error handling, and protected downloads.
- Top navigation, help menu, workflow selector, settings token display, dark-mode CSS, POS logo proxy, POS print alerts, and stored-file/backup routes.
- Items, inventory, recipes, batches, containers, firearms, ballistics, audit, QR, and export route proxy payloads.
- Template behavior for category-specific item fields, grouped inventory, active-lot dialogs, recipe source upload fields, component role derivation, recipe filters/sorts, recipe performance charts, Garmin-locked fields, batch QA/lifecycle controls, production-loss and return forms, clickable rows, and container mixed-batch controls.
- Static JavaScript for inventory prompts, recipe source fields, ballistics weather helpers, recipe performance charts, batch lifecycle submit, print-error acknowledgement, and clickable-row hover handling.

Ballistics

Ballistics tests cover:

- Public health endpoint and bearer-token validation.
- Open-Meteo geocode and current-weather normalization.
- Trajectory corrections using `py-ballisticcalc`.
- G1/G7 input validation, realistic velocity loss, zeroing behavior, wind correction behavior, environment density effects, implausible-input warnings, and solver failure mapping.
- Nginx and production-compose routing for the ballistics hostname.
- Pinned `py-ballisticcalc` dependency.

POS Printing

POS tests cover:

- Health status and dry-run mode.
- Batch-created and batch-produced ESC/POS rendering.
- MCP print marker placement.
- Bullet ballistic data on receipts.
- QR sections, logo fallback handling, text/image test jobs, bounded dry-run job log, and printer transport failure responses.
- Direct print script refusal unless the service is dry-run or `--allow-real-printer` is explicitly supplied.

MCP Bridge

MCP tests cover:

- JSON-RPC initialize, ping, parse errors, unknown methods, and newline-delimited stdio responses.
- Tool listing and argument validation.
- Login, logout, whoami, token setup, bearer-token API calls, path normalization, content-type handling, and request error mapping.
- Generic API bridge tools.
- Workflow tools for sourced recipe creation, combined recipe/batch/container workflows, rollback behavior, storage-not-satisfied reporting, and explicit POS batch print events.

Selenium Browser Workflow

The Selenium workflow covers a full .357 Magnum browser path:

- Registration, failed login, successful login, logout, and relogin.
- Item and inventory creation.
- Recipe creation, source handling, approval, public sharing, and duplicate core-component prevention.
- Successor-lot promotion through batch production.
- Batch creation, QA, production, and performance entry.

- Garmin FIT import.
- Container creation, assignment, overfill rejection, mixed-batch acknowledgement, storage state changes, depletion, and audit checks.

Acceptance Checks

Acceptance item	Verification evidence
Test inventory is current.	<code>docker compose run --rm storage pytest --collect-only -q</code> completes and includes all listed modules.
Non-Selenium tests pass.	<code>POS_PRINT_DRY_RUN=true docker compose run --rm storage pytest -q</code> exits <code>0</code> and reports the Selenium test skipped by default.
Selenium tests pass.	The Selenium command exits <code>0</code> with the E2E test passing.
Browser workflow is observable when needed.	Visible-mode run can be watched at <code>http://localhost:7900</code> .
Failures are triaged.	Record node ID, command, output, browser page text if supplied, relevant logs, and git revision/local diff.
Documentation is reproducible.	Regenerate HTML with <code>pandoc docs/Test_Execution_Guide.md --standalone --toc --css=/home/swiseman/repositories/reloading_app/docs/Test_Execution_Guide.css -o /tmp/Test_Execution_Guide.html</code> , then PDF with <code>wkhtmltopdf --enable-local-file-access --orientation Landscape --page-size Letter --margin-top 8mm --margin-right 8mm --margin-bottom 8mm --margin-left 8mm /tmp/Test_Execution_Guide.html docs/Test_Execution_Guide.pdf</code> .